

PECL1 - SIMULACIÓN DE UNA CAFETERÍA

Paradigmas de Programación

Ingeniería de Sistemas de Información

(Curso 2025/2026 - Convocatoria Ordinaria)



13 de diciembre de 2025

Mauro Sánchez - [REDACTED] | Sergio Pavón - [REDACTED]

Índice

Análisis de Alto Nivel.....	2
1.1. Descripción de Actores y Ciclos de Vida.....	2
1.2. Gestión de Zonas y Restricciones de Aforo.....	2
1.3. Gestión de Recursos Compartidos y Datos.....	3
1.4. Interfaz de Usuario y Control.....	3
1.5. Programación Distribuida (Módulo Remoto).....	3
Diseño General del Sistema.....	4
Arquitectura y Modelo de Concurrencia.....	4
Estrategia de Sincronización.....	4
Descripción Detallada de Clases.....	5
PracticaCafeteria - Clase Main.....	5
Clase Log.....	5
Clase Mostrador.....	6
Clase Despensa.....	6
Clase Caja.....	7
Clase ControlAforos.....	8
Clase Cliente.....	9
Clase Cocinero.....	9
Clase Vendedor.....	10
Clase InterfazCafeteria.....	10
Clase ICafeteriaRemota.....	11
Clase CafeteriaRemotaImpl.....	12
Clase ClienteRemoto.....	13
Diagrama de Clases.....	14
Anexo: Código Fuente.....	15

Análisis de Alto Nivel

El objetivo de este proyecto es simular el funcionamiento de una cafetería mediante técnicas de programación concurrente y programación distribuida en Java. El sistema que hemos creado se encarga de modelar un entorno donde múltiples actores independientes compiten por recursos limitados bajo estrictas reglas de sincronización, respetando tiempos de ejecución y los aforos predefinidos.

1.1. Descripción de Actores y Ciclos de Vida

El sistema requiere de la generación concurrente de tres tipos de actores, modelados como hilos, cuya creación debe ser escalonada para evitar la saturación inicial del sistema.

- **Clientes (8.000 hilos):** Son los consumidores del sistema. Se generan en intervalos de 1 a 3 segundos. Su ciclo de vida simula un flujo real: comienzan en el parque, se desplazan a la entrada, hacen cola, acceden al **Mostrador** para pedir (1-3 cafés y 0-4 rosquillas), pagan en la **Caja** y consumen los productos antes de finalizar su ejecución, en el **Área de Consumición**.
- **Cocineros (500 hilos):** Son los productores de los alimentos. Se generan en intervalos de 1 a 2 segundos. Su función es generar stock en la **Cocina** (2-5 cafés o 4-8 rosquillas por ciclo) y transportarlo a la **Despensa**.
- **Vendedores (500 hilos):** Son los intermediarios. Se generan en intervalos de 0,5 a 2,5 segundos. Su labor es trasladar productos desde la **Despensa** hasta el **Mostrador**, reponiendo las estanterías para que los clientes puedan ser atendidos.

1.2. Gestión de Zonas y Restricciones de Aforo

Uno de los requisitos funcionales críticos es el control de acceso a las distintas zonas de la cafetería para evitar saturaciones y condiciones de carrera. Para ello, hemos implementado estos límites mediante semáforos, respetando así los siguientes aforos:

- **Entrada a Cafetería/Espera:** Existe un buffer de entrada limitado a **20 personas** esperando para acceder a la zona del mostrador. Si está lleno, los clientes deberán esperar fuera.
- **Mostrador (Zona Clientes):** Solo **5 clientes** pueden estar siendo atendidos simultáneamente seleccionando los productos.
- **Mostrador (Zona Vendedores):** Se permite la presencia simultánea de hasta **20 vendedores** reponiendo las estanterías.
- **Caja:** El aforo está limitado a **10 clientes**, correspondientes a las 10 máquinas de autopago que hay disponibles. La atención sigue el algoritmo **FIFO** (First-In, First-Out) que se basa en el orden de llegada.
- **Área de Consumición:** Limitada a **30 personas**. Los clientes deben esperar si todas las mesas están ocupadas.
- **Cocina:** Tiene una capacidad máxima para **100 cocineros** trabajando simultáneamente.
- **Despensa:** El aforo es compartido pero limitado a **50 cocineros** y **50 vendedores** en su interior al mismo tiempo.
- **Zonas sin límites:** El Parque y la Sala de Descanso tienen capacidad ilimitada.

1.3. Gestión de Recursos Compartidos y Datos

El sistema gestiona la información crítica accedida y modificada por múltiples hilos simultáneamente, para ello hacemos uso de la protección de exclusión mutua mediante cerrojos y monitores:

- **Inventario (Buffers):** Las estanterías de cafés y rosquillas actúan como buffers de almacenamiento intermedio.
 - En la **Despensa**, los cocineros depositan y los vendedores retiran.
 - En el **Mostrador**, los vendedores depositan y los clientes retiran.
 - Hacemos uso de las variables de condición para gestionar la espera si los buffers están vacíos o si no hay stock suficiente.
- **Recaudación:** Se trata de una variable crítica que almacena los ingresos totales. Deben actualizarse de forma atómica cada vez que un cliente confirma su pago en la **Caja**.
- **Log del Sistema:** Generamos un fichero "*evolucion_cafeteria.txt*" que registra cronológicamente todos los eventos relevantes del sistema (movimientos, transacciones), incluyendo una marca de tiempo precisa. Este log actúa como un recurso compartido centralizado protegido por su propio cerrojo para evitar las escrituras entremezcladas.

1.4. Interfaz de Usuario y Control

El sistema cuenta con una interfaz gráfica (GUI) basada en Swing que nos permite visualizar en tiempo real la ubicación de los actores (listas de IDs de cada zona) y el estado numérico de los inventarios. Además, incluimos un requisito funcional de control crítico: un botón para **Pausar/Reanudar** la ejecución de la simulación. Esta funcionalidad la implementamos mediante una variable de condición global que bloquea a todos los hilos activos cuando el estado es "pausado".

1.5. Programación Distribuida (Módulo Remoto)

Para cumplir con los requisitos de la parte 2 de monitorización remota, hemos hecho uso en el sistema el **Java RMI (Remote Method Invocation)**:

- **Servidor:** La aplicación principal es **PracticaCafeteria** básicamente es el main, se encarga de ejecutar la lógica de simulación, exponiendo un objeto remoto que implementa la interface **ICafeteriaRemota**.
- **Cliente Remoto:** La aplicación externa **ClienteRemoto** se conecta al registro RMI del servidor para consultar el estado de la cafetería.
- **Modelo de Comunicación:** El cliente remoto se encarga de actualizar la información de aforos, stock y recaudación cada segundo (`Thread.sleep(1000)`). También tiene la capacidad de mando para detener o reanudar la simulación remotamente.

Diseño General del Sistema

Arquitectura y Modelo de Concurrency

El sistema se ha realizado bajo una arquitectura concurrente y distribuida diseñada para simular el funcionamiento real de una cafetería. La estructura de la aplicación se divide en dos partes independientes que se comunican a través de la red: un servidor central, que aloja la lógica de negocio y se encarga del ciclo de vida de los hilos, y un cliente remoto destinado a la monitorización y control externo.

El núcleo del diseño se fundamenta en un patrón de **Productores y Consumidores** anidados que funciona como una tubería de procesamiento. En una primera etapa, los hilos de tipo **Cocinero** actúan como productores generando suministros que son depositados en la **Despensa**, un recurso compartido donde los **Vendedores** ejercen el rol de consumidores al extraer dicho suministro. Después, se inicia una segunda etapa en la que los **Vendedores** invierten su rol, convirtiéndose en productores al depositar los pedidos preparados en el **Mostrador** y poniéndolos a disposición de los **Cientes**, quienes cierran el ciclo.

Estrategia de Sincronización

Para garantizar la integridad de los datos y la coordinación eficiente entre los miles de hilos activos, hemos implementado una estrategia basada en las herramientas de alto nivel del paquete **java.util.concurrent**.

El control de los recursos físicos limitados se gestiona mediante el uso extensivo de los **Semáforos**. Estos mecanismos se encargan de regular el aforo en las distintas zonas de la cafetería, impidiendo la saturación del sistema. Por ejemplo, los semáforos pueden restringir el número máximo de cocineros y comensales permitidos en sus respectivas áreas, controlan la longitud de la cola de espera dentro del local, limitan la atención simultánea en la barra y gestionan la disponibilidad de las terminales de pago en la caja.

Para asegurar la consistencia de los datos compartidos, hemos empleado los **Cerros** que nos garantizan la exclusión mutua en las secciones críticas. Cada monitor del sistema protege sus variables de estado mediante estos cerros. De esta forma, nos aseguramos que las operaciones de lectura y escritura sean atómicas, evitando las condiciones de carrera y garantizando que las listas que alimentan la interfaz gráfica reflejan siempre un estado coherente.

La coordinación lógica entre los distintos actores lo resolvemos mediante las **Variables de Condición** asociadas a los cerros. Este mecanismo nos permite la sincronización condicional, facilitando que los hilos suspendan su ejecución voluntariamente cuando no pueden seguir. Gracias a esto, los vendedores pueden esperar eficientemente si la despensa está vacía y los clientes esperan si no hay stock en el mostrador, activándose únicamente cuando reciben una notificación de reposición. También, este sistema nos ayuda en la funcionalidad de pausa global, permitiendo detener y reanudar la ejecución de todos los actores de manera sincronizada.

Finalmente, al proyecto le incorporamos mecanismos específicos para necesidades puntuales. El sistema de registro (*Log*) utiliza la sincronización nativa mediante bloques **synchronized** para serializar el acceso al archivo de texto, asegurando una traza cronológica. Por su parte, la comunicación entre procesos se articula mediante **Java RMI**, tecnología que permite al cliente remoto invocar métodos del servidor de forma transparente, logrando monitorizar el estado de los monitores y cerrojos en tiempo real a través de la red.

Descripción Detallada de Clases

El sistema que hemos elaborado se estructura en un conjunto de clases que colaboran entre sí, siguiendo el paradigma de Orientación a Objetos y el modelo de monitores para la gestión de la concurrencia. A continuación, detallamos todas las clases que hemos implementado, con sus atributos y métodos.

PracticaCafeteria - Clase Main

Actúa como el punto de entrada y de arranque del sistema. Su función principal es inicializar la infraestructura de la aplicación, establecer la conectividad de red y lanzar los hilos de ejecución.

Atributos: Carece de atributos de clase o instancia. Hace uso de variables locales en el método main para realizar la inyección de dependencias entre los componentes (GUI, Log y Recursos Compartidos).

Métodos: **main(String[] args)**. Se encarga de ejecutar la secuencia de arranque del sistema:

- **Inicialización:** Instancia el Log y la InterfazCafeteria.
- **Recursos Compartidos:** Crea los monitores (ControlAforos, Despensa, Mostrador, Caja).
- **Red RMI:** Registra el objeto remoto CafeteriaRemotalmpl en el puerto 1099 para permitir el monitoreo externo.
- **Hilos:** Genera y lanza, mediante bucles con retardos aleatorios, los hilos de Cocinero (500), Vendedor (500) y Cliente (8000).

Clase Log

Es el componente encargado de la trazabilidad del sistema. Implementamos un monitor de escritura que garantiza la exclusión mutua para que los múltiples hilos puedan registrar los eventos concurrentemente en un archivo de texto sin corromper los datos.

Atributos:

- **PrintWriter writer:** Muestra el flujo de salida hacia el archivo evolucion_cafeteria.txt.
- **DateTimeFormatter formatter:** Se encarga de definir el patrón temporal "yyyy-MM-dd HH:mm:ss".

Métodos:

- **Log() (Constructor):** Inicializa el flujo de escritura hacia el archivo en disco.
- **escribirLog(String mensaje):** Es un método synchronized. Obtiene la hora actual, formatea el mensaje y luego lo escribe en el archivo, forzando al guardado inmediato para evitar pérdida de datos.

Clase Mostrador

Es el recurso compartido central que actúa como **buffer** entre los Vendedores y los Clientes. Se encarga de gestionar el stock de productos listos y regula el flujo de entrada de los clientes mediante un sistema de etapas (cola y barra) que es controlado por semáforos.

Atributos:

- **Stock:** int cafes e int rosquillas.
- **Concurrencia:** Lock cerrojo (ReentrantLock) para exclusión mutua y Condition hayStock para la espera eficiente de los productos.
- **Semáforos:** colaEsperaMostrador (20) y aforoAtencionClientes (5) para limitar la capacidad en las distintas zonas.
- **Listas:** ArrayList para el seguimiento visual en la GUI (listClientesCola, listClientesAtencion, etc.).

Métodos:

- **Gestión de Vendedores:**
 - **entrarVendedor()** y **salirVendedor():** Controlan el acceso y registro en GUI.
 - **reponer():** Incrementa el stock bajo bloqueo y ejecuta signalAll() para despertar a los clientes que esperan la comida.
- **Gestión de Clientes:**
 - **entrarColaEspera():** Gestiona el acceso al local (semáforo de 20).
 - **accederMostrador():** Gestiona el paso de la cola a la barra (semáforo de 5), liberando hueco en la cola anterior.
 - **pedirYRecoger():** Espera condicional (await) si no hay stock; decrementa el inventario cuando es posible.
 - **salirMostrador():** Libera el hueco en la barra.
- **Consultas:** Los métodos *get* están protegidos por cerrojo para uso seguro desde RMI.

Clase Despensa

Es el recurso compartido que actúa como un almacén primario, se encarga de conectar la producción de los **Cocineros** con la recolección de los **Vendedores**. En esta implementamos un modelo Productor - Consumidor donde los cocineros depositan los productos y los vendedores esperan si el almacén está vacío.

Atributos:

- **Stock:** int cafes e int rosquillas.

- **Semáforos de Aforo:** aforoCocineros (50) y aforoVendedores (50) para limitar el personal simultáneo en el área de cocina/almacén.
- **Concurrencia:** Lock cerrojo para proteger el acceso al stock y Condition hayComida para gestionar la espera de vendedores cuando el stock es cero.
- **Listas:** ArrayList para visualizar qué cocineros y vendedores están dentro del recurso.

Métodos:

- **Gestión de Cocineros (Productores):**
 - **entrarCocinero()** y **salirCocinero()**: Sirven para gestionar el semáforo de aforo y actualizar la lista de presencia.
 - **dejarProductos()**: Se encarga de incrementar el stock de forma segura y ejecuta **hayComida.signalAll()** que se usa para notificar a los vendedores bloqueados que ya hay suministros disponibles.
- **Gestión de Vendedores (Consumidores):**
 - **entrarVendedor()** y **salirVendedor()**: Sirven para gestionar el semáforo de aforo y actualizar la lista de presencia.
 - **cogerProductos()**: Es un método complejo que implementamos para una espera condicional (await) si el stock total es 0. Si hay stock, este calcula cuánto puede llevarse (total solicitado o lo que quede disponible) y devolviendo un array con la cantidad real obtenida.
- **Consultas:** Usamos los métodos *get* sincronizados para la monitorización externa (RMI).

Clase Caja

Es el recurso compartido que representa la etapa final del ciclo de vida del cliente. Simulamos una zona de pago con una capacidad limitada (máquinas de autopago) y se encarga de centralizar la contabilidad financiera de la simulación de manera segura ante los accesos concurrentes.

Atributos:

- **Contabilidad:** recaudacionTotal (double) es la variable acumuladora de ingresos.
- **Semáforo:** aforoCaja (10) limita el número de clientes que pueden estar pagando simultáneamente (simulando 10 terminales).
- **Concurrencia:** Lock cerrojo (ReentrantLock) se encarga de proteger la variable de recaudación y las listas de seguimiento.
- **Listas:** listEsperandoCaja (cola previa) y listEnCaja (usando máquina) lo implementamos para la visualización en el GUI.

Métodos:

- **Flujo de Pago:**
 - **llegarColaCaja(String id)**: Se encarga de registrar al cliente en la cola de espera protegiendo la lista con el cerrojo.

- **usarMaquinaPago(String id):** Solicita un permiso al semáforo (acquire). Si las 10 máquinas están ocupadas, el hilo se bloquea. Al entrar, actualiza las listas de la GUI.
- **confirmarPago(String id, double importe):** Es la Sección Crítica donde se suma el importe a recaudacionTotal, se escribe en el Log y se libera el semáforo (release), permitiendo el paso a otro cliente.
- **Consultas:** Los métodos *get* (getRecaudacion, getCantidadClientes) están protegidos por cerrojo para garantizar las lecturas consistentes desde RMI.

Clase ControlAforos

Es el monitor general encargado de la gestión espacial de la simulación. Se centra en controlar el acceso a las zonas comunes y restringidas (Cocina, Área de Consumición) y mantiene el registro visual de la ubicación de todos los actores del sistema (Parque, Salas de Descanso, etc.).

Atributos:

- **Semáforos de Capacidad:** semCocina (100) y semAreaConsumicion (30) para imponer límites físicos en dichas zonas.
- **Concurrencia:** Lock cerrojoListas (ReentrantLock) utilizado exclusivamente para garantizar la seguridad de hilos al modificar las listas que alimentan la interfaz gráfica.
- **Listas de Seguimiento:** Múltiples ArrayList (listParque, listCocina, listSalaDescanso..., etc.) que almacenan los IDs de los hilos presentes en cada ubicación.

Métodos:

- **Zonas sin Límite (Parque, Salas de Descanso):**
 - **Métodos entrar/salir...:** Se encargan de proteger la modificación de las listas con cerrojoListas y actualiza la GUI. No bloquean el flujo de ejecución del hilo más allá de la exclusión mutua para la escritura.
- **Zonas con Límite (Cocina, Consumo):**
 - **entrarCocina() / entrarConsumir():** Ejecutan acquire() sobre el semáforo correspondiente. Si la zona está llena (100 cocineros o 30 comensales), el hilo solicitante se bloquea hasta que haya espacio.
 - **salirCocina() / salirConsumir():** Ejecutan release(), liberando un permiso y permitiendo la entrada de un hilo que está en espera.
- **Gestión de Espera:**
 - **esperarMesa():** Es un método transitorio que se encarga de colocar al cliente en una lista de espera visual antes de intentar adquirir el semáforo de consumición.
- **Consultas:** Los métodos *get* para RMI devuelven el tamaño actual de las listas bajo protección del cerrojo.

Clase Cliente

Es la clase que hereda del hilo, representando al consumidor dentro de la simulación. Su ciclo de vida es lineal y atraviesa secuencialmente todos los recursos compartidos, simulando el comportamiento humano (esperar, pedir, pagar, comer) con retardos aleatorios.

Atributos:

- **Identidad:** id (String) único para su trazabilidad.
- **Dependencias:** Tiene referencias a ControlAforos, Mostrador, Caja e InterfazCafeteria para interactuar con el entorno.
- **Aleatoriedad:** Hacemos uso de Random rand para generar los tiempos de espera y decidir la cantidad de productos a consumir.

Métodos:

- **esperar(int min, int max):** Es un método auxiliar que simula el transcurso del tiempo (Thread.sleep). Incluye una llamada crítica a gui.comprobarPausa(), lo que permite detener el hilo si el usuario pausa la simulación desde la interfaz.
- **run():** Define el flujo principal del hilo:
 1. **Parque:** Entra y pasea (interacción con ControlAforos).
 2. **Mostrador:** Entra en cola, pasa a la barra, selecciona un pedido aleatorio (0-3 cafés, 0-4 rosquillas) y espera a recibirlo (interacción con Mostrador).
 3. **Pago:** Se dirige a la caja, usa una máquina de autopago y abona el importe calculado (1.50€/café, 2.50€/rosquilla) (interacción con Caja).
 4. **Consumición:** Busca mesa en el área de descanso y consume los productos durante un tiempo determinado (interacción con ControlAforos).
 5. **Salida:** Abandona el sistema y el hilo termina.

Clase Cocinero

Es la clase que hereda del hilo, que desempeña el rol de productor en el sistema. A diferencia del cliente, su ciclo de vida es infinito hacemos uso de while-true, simulando una jornada laboral continua donde alterna entre descansar, cocinar y reponer la despensa.

Atributos:

- **Identidad:** id (String) para su trazabilidad.
- **Dependencias:** Tiene referencias a ControlAforos (para acceder a la cocina y sala de descanso) y Despensa (para depositar lo producido).
- **Control:** Usamos la InterfazCafeteria gui para gestionar las pausas de la simulación.
- **Aleatoriedad:** Y al igual que Clientes usamos Random rand para variar tiempos de cocinado y cantidad de producción.

Métodos:

- **esperar(int min, int max):** Es un método auxiliar que gestiona el Thread.sleep y verifica en cada paso si la simulación ha sido pausada desde la GUI.

- **run():** Ejecuta un bucle infinito con la rutina del empleado:
 1. **Descanso:** Ingresa a la sala de descanso de cocineros y permanece allí un tiempo aleatorio (interacción con ControlAforos).
 2. **Producción (Cocina):** Se desplaza a la cocina. Debe adquirir el semáforo de cocina (max 100). Una vez dentro, simula el tiempo de preparación.
 3. **Reposición (Despensa):** Se dirige a la despensa. Debe adquirir el semáforo de despensa (max 50). Deposita el lote fabricado mediante `despensa.dejarProductos()`, lo cual notifica a los vendedores, y regresa al inicio del ciclo.

Clase Vendedor

Es la última clase que hereda de Thread. Representa un agente con un rol híbrido en el esquema de concurrencia: actúa como consumidor respecto a la **Despensa** y como productor respecto al **Mostrador**. Su función es transportar los productos procesados desde el almacén hasta la barra de atención al cliente.

Atributos:

- **Identidad:** id (String) para su trazabilidad.
- **Dependencias:** Tiene referencias a Despensa (origen), Mostrador (destino) y ControlAforos (para zona de descanso).
- **Control:** Usamos InterfazCafeteria gui para respetar las pausas del sistema.
- **Aleatoriedad:** Random rand para determinar la cantidad de producto que intenta retirar y los tiempos de traslado.

Métodos:

- **esperar(int min, int max):** Gestión del tiempo de simulación y pausa global.
- **run():** Implementa un bucle infinito que define la rutina del camarero/vendedor:
 1. **Descanso:** Ingresa a la sala de descanso de vendedores (interacción visual con ControlAforos).
 2. **Abastecimiento (Despensa):** Entra a la despensa.
 3. **Reposición (Mostrador):** Se traslada al mostrador.
 4. **Reinicio:** Regresa a la sala de descanso para comenzar el ciclo nuevamente.

Clase InterfazCafeteria

Esta clase hereda de `javax.swing.JFrame` y representa la interfaz gráfica de usuario (GUI) de la aplicación. Su responsabilidad principal es visualizar en tiempo real el estado de todos los recursos compartidos y la ubicación de los actores (clientes, cocineros, vendedores) mediante áreas de texto y contadores. Adicionalmente, implementamos la lógica de **Pausa Global** de la simulación, que actúa como un monitor que nos permite detener y reanudar la ejecución de todos los hilos activos de manera sincronizada.

Atributos:

- **Control de Estado (Concurrencia):**
 - **private boolean pausado:** Es la bandera (*flag*) que indica si la simulación está detenida.
 - **Lock cerrojoPausa (ReentrantLock):** Nos garantiza la exclusión mutua para modificar el estado de pausa.
 - **Condition condPausa:** Es una variable de condición asociada al cerrojo, es utilizada para bloquear a los hilos cuando la pausa está activa y despertarlos (*signalAll*) cuando se reanuda.

Métodos:

- **Inicialización:**
 - **InterfazCafeteria() (Constructor):** Configuramos las propiedades de la ventana (título, tamaño 1250x850, centrado) e invocamos a *initComponents()*.
 - **initComponents():** Este método es generado automáticamente por el IDE que instancia, configura y distribuye geométricamente todos los componentes visuales dentro del *JFrame*.
- **Gestión de Eventos:**
 - **btnPausarActionPerformed(ActionEvent evt):** Es el manejador del evento de clic del botón. Invoca al método *cambiarEstadoPausa()*.
- **Lógica de Pausa (Monitor):**
 - **cambiarEstadoPausa():** Adquiere el *cerrojoPausa*. Luego, invierte el valor del booleano *pausado* actualizando el texto del botón ("Pausar" / "REANUDAR"). Si reanudamos la simulación (*pausado == false*), se ejecuta la *condPausa.signalAll()* que despierta a todos los hilos que estaban bloqueados esperando.
 - **comprobarPausa():** Este método es invocado periódicamente por todos los hilos del sistema (Cliente, Cocinero, Vendedor) dentro de sus bucles de ejecución.

Clase ICafeteriaRemota

Esta interfaz define el contrato de comunicación para el sistema **RMI (Remote Method Invocation)**. Su propósito es exponer un conjunto limitado y seguro de operaciones para que las aplicaciones externas puedan monitorizar el estado de la cafetería (consultar aforos, stocks y recaudación) y controlar su ejecución (pausar/reanudar) sin acceder directamente a la lógica interna de los monitores.

Atributos: Como interfaz pura, no hay atributos de estado ni constantes en este diseño. Solo se declaran las firmas de los métodos que la implementación del servidor debe proveer.

Métodos: Todos los métodos que hemos definido lanzan la excepción obligatoria *java.rmi.RemoteException* para gestionar los posibles fallos en la comunicación de red. Se agrupan funcionalmente de la siguiente manera:

- **Monitorización de Clientes:**
 - **obtenerClientesParque()**, **obtenerClientesMostrador()**, **obtenerClientesCaja()**, **obtenerClientesConsumicion()**: Devuelven los valores enteros (int) que representan la cantidad instantánea de clientes ubicados en cada una de las zonas clave del sistema.
- **Monitorización de Personal:**
 - **obtenerCocinerosCocina()**, **obtenerCocinerosDespensa()**: Nos devuelve el número de cocineros activos en sus zonas de trabajo.
 - **obtenerVendedoresDespensa()**, **obtenerVendedoresMostrador()**: Nos devuelve el número de vendedores activos en sus respectivas zonas.
 - **obtenerPersonalDescanso()**: Nos devuelve un entero con la suma total de cocineros y vendedores que se encuentran actualmente en sus respectivas salas de descanso.
- **Monitorización de Inventario (Stock):**
 - **obtenerStockDespensaCafe()**, **obtenerStockDespensaRosquillas()**: Consultan la cantidad de materia prima disponible en el almacén principal.
 - **obtenerStockMostradorCafe()**, **obtenerStockMostradorRosquillas()**: Consultan la cantidad de producto preparado disponible en la barra para la venta inmediata.
- **Monitorización Financiera:**
 - **obtenerRecaudacion()**: Nos devuelve un valor de tipo double con el saldo total acumulado en la Caja hasta el momento de la consulta.
- **Control de Ejecución:**
 - **pausarOReanudarServidor()**: Es un método de acción (tipo void) que nos permite invocar remotamente el mecanismo de pausa global de la interfaz gráfica (`gui.cambiarEstadoPausa()`), sincronizando el estado del servidor con el comando externo.

Clase CafeteriaRemotaImpl

Esta clase implementa la interfaz `ICafeteriaRemota` y extiende de `java.rmi.server.UnicastRemoteObject`. Su función es actuar como la implementación del objeto remoto en el lado del servidor. Funciona como un patrón Fachada (Facade) para la red: no contiene lógica de negocio propia, sino que centraliza y delega las peticiones externas hacia los monitores internos correspondientes (`ControlAforos`, `Mostrador`, etc.), permitiéndonos que un cliente remoto consulte el estado de la simulación de forma segura y transparente.

Atributos: Mantenemos las referencias privadas a todas las instancias de los recursos compartidos y la interfaz gráfica, siendo inicializadas vía constructor (Inyección de Dependencias):

- **ControlAforos aforos**, **Despensa despensa**, **Mostrador mostrador**, **Caja caja**: Para consultar datos en tiempo real.
- **InterfazCafeteria guiServer**: Para ejecutar acciones de control (pausa).

Métodos:

- **Constructor:** Invocamos al constructor de la superclase (`super()`) para exportar el objeto RMI y asigna las referencias de los recursos.

- **Métodos de Consulta (obtener...):** Cada método de la interfaz se implementa delegando la llamada al método *getter* correspondiente del monitor específico (ej. obtenerClientesParque, aforos.getCantidadParque()).
- **pausarOReanudarServidor():** Redirige la orden remota al método guiServer.cambiarEstadoPausa(), logrando que la acción de pausar desde un cliente remoto tenga el mismo efecto que pulsar el botón físico en el servidor.

Clase ClienteRemoto

Esta clase representa una aplicación cliente independiente que se ejecuta en una JVM distinta a la del servidor. Hereda de javax.swing.JFrame y actúa como un **Monitor Remoto**. Su responsabilidad es conectarse al servidor mediante RMI para visualizar en tiempo real el estado de la simulación y enviar comandos de control (pausa). Actualizar la interfaz gráfica periódicamente con los datos más recientes del servidor.

Atributos:

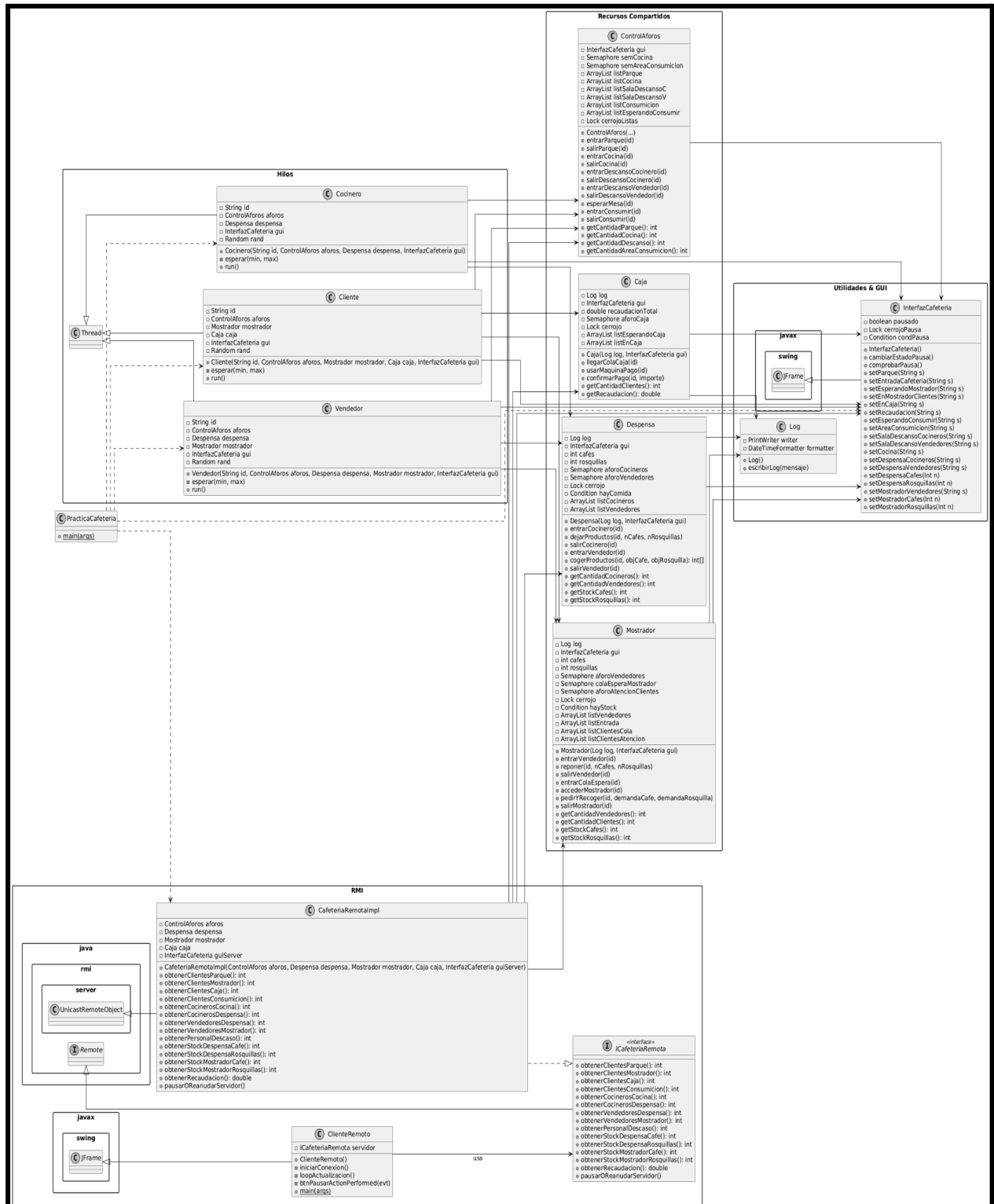
- **Proxy RMI:** private ICafeteriaRemota servidor. Es la referencia local (Stub) al objeto remoto. A través de este atributo se invocan los métodos del servidor como si fueran locales.
- **Logging:** Logger logger para el registro de excepciones del cliente.
- **Componentes Visuales:** Campos de texto (JTextField) para mostrar los valores recibidos (aforos, stocks, caja) y un botón (btnPausar) para enviar la orden de detención.

Métodos:

- **iniciarConexion():** Realiza la búsqueda del servicio RMI mediante Naming.lookup("//localhost/Cafeteria"). Si la conexión es exitosa, inicializa el atributo servidor y lanza un hilo secundario encargado de la actualización de datos.
- **loopActualizacion():** Es el método ejecutado por un hilo en segundo plano.
 - Este ejecuta un bucle infinito que consulta todos los métodos *getter* de la interfaz remota (obtenerClientesParque, obtenerRecaudacion, etc.) cada 1 segundo.
 - **Seguridad de Hilos (Swing):** Utiliza SwingUtilities.invokeLater(...) para las actualizaciones, evitando errores de concurrencia en la interfaz gráfica.
- **btnPausarActionPerformed():** Captura el evento del botón y delega la acción llamando a servidor.pausarOReanudarServidor(), lo que detiene la simulación en el proceso servidor.
- **main():** Punto de entrada de la aplicación cliente. Configura el aspecto visual y hace visible la ventana.

Diagrama de Clases

A continuación, mostramos el diagrama de clases de nuestro código, el cual lo hemos elaborado con la herramienta PlantText. Si no fuera posible ver el contenido de la imagen con claridad, adjuntamos en el zip el diagrama en formato PDF.



Anexo: Código Fuente

El anexo de nuestro código es el siguiente:

```
package pdp.practicacafeteria;

import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;

/**
 *
 * @author sergiopavon
 */
public class PracticaCafeteria {

    public static void main(String[] args) {
        // Inicializar Logger y GUI
        Log log = new Log();
        InterfazCafeteria gui = new InterfazCafeteria();
        gui.setVisible(true);

        // Inicializar Recursos Compartidos
        ControlAforos aforos = new ControlAforos(gui);
        Despensa despensa = new Despensa(log, gui);
        Mostrador mostrador = new Mostrador(log, gui);
        Caja caja = new Caja(log, gui);

        // Iniciar servidor RMI
        try {
            CafeteriaRemotaImpl objetoRemoto = new CafeteriaRemotaImpl(aforos,
                despensa, mostrador, caja, gui);
            LocateRegistry.createRegistry(1099); // Para RMI, puerto 1099
            Naming.rebind("//localhost/Cafeteria", objetoRemoto);

            System.out.println("Servidor RMI Activo y esperando clientes...");

        } catch (Exception e) {
            System.err.println("Error al iniciar RMI: " + e.getMessage());
        }

        // Creación de hilos cocineros
        new Thread(() -> {
            for (int i = 1; i <= 500; i++) {
                String id = String.format("B-%04d", i);
                Cocinero c = new Cocinero(id, aforos, despensa, gui);
                c.start();
                try {
                    // Intervalo 1 a 2 segundos
                    Thread.sleep((int)(1000 + 1000 * Math.random()));
                } catch (InterruptedException e) {}
            }
        }).start();

        // Creación de hilos vendedores
        new Thread(() -> {
            for (int i = 1; i <= 500; i++) {
                String id = String.format("V-%04d", i);
                Vendedor v = new Vendedor(id, aforos, despensa, mostrador, gui);
            }
        }).start();
    }
}
```



```

        v.start();
        try {
            // Intervalo 0.5 a 2.5 segundos
            Thread.sleep((int)(500 + 2000 * Math.random()));
        } catch (InterruptedException e) {}
    }
    }).start();

    // Creación de hilos clientes
    new Thread(() -> {
        for (int i = 1; i <= 8000; i++) {
            String id = String.format("C-%04d", i);
            Cliente c = new Cliente(id, aforos, mostrador, caja, gui);
            c.start();
            try {
                // Intervalo 1 a 3 segundos
                Thread.sleep((int)(1000 + 2000 * Math.random()));
            } catch (InterruptedException e) {}
        }
    }).start();
}
}
}

```

```

package pdp.practicacafeteria;

import java.util.ArrayList;
import java.util.concurrent.Semaphore;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

/**
 *
 * @author sergiopavon
 */
public class ControlAforos {
    private InterfazCafeteria gui;

    // Semáforos de capacidad
    private Semaphore semCocina = new Semaphore(100);
    private Semaphore semAreaConsumicion = new Semaphore(30);

    // Listas para visualización
    private ArrayList<String> listParque = new ArrayList<>();
    private ArrayList<String> listCocina = new ArrayList<>();
    private ArrayList<String> listSalaDescansoC = new ArrayList<>();
    private ArrayList<String> listSalaDescansoV = new ArrayList<>();
    private ArrayList<String> listConsumicion = new ArrayList<>();
    private ArrayList<String> listEsperandoConsumir = new ArrayList<>();

    // Cerrojo para proteger las listas
    private Lock cerrojoListas = new ReentrantLock();

    public ControlAforos(InterfazCafeteria gui) {
        this.gui = gui;
    }
}

```

```
// Métodos para manejar las listas
public void entrarParque(String id) {
    cerrojoListas.lock();
    try {
        listParque.add(id);
        gui.setParque(listParque.toString());
    } finally {
        cerrojoListas.unlock();
    }
}

public void salirParque(String id) {
    cerrojoListas.lock();
    try {
        listParque.remove(id);
        gui.setParque(listParque.toString());
    } finally {
        cerrojoListas.unlock();
    }
}

// Cocina, aforo máximo = 100
public void entrarCocina(String id) throws InterruptedException {
    semCocina.acquire();
    cerrojoListas.lock();
    try {
        listCocina.add(id);
        gui.setCocina(listCocina.toString());
    } finally {
        cerrojoListas.unlock();
    }
}

public void salirCocina(String id) {
    cerrojoListas.lock();
    try {
        listCocina.remove(id);
        gui.setCocina(listCocina.toString());
    } finally {
        cerrojoListas.unlock();
    }
    semCocina.release();
}

// Sala de descanso, sin límite de aforo
public void entrarDescansoCocinero(String id) {
    cerrojoListas.lock();
    try {
        listSalaDescansoC.add(id);
        gui.setSalaDescansoCocineros(listSalaDescansoC.toString());
    } finally {
        cerrojoListas.unlock();
    }
}

public void salirDescansoCocinero(String id) {
    cerrojoListas.lock();
    try {
        listSalaDescansoC.remove(id);
    }
}
```

```

        gui.setSalaDescansoCocineros(listSalaDescansoC.toString());
    } finally {
        cerrojoListas.unlock();
    }
}

public void entrarDescansoVendedor(String id) {
    cerrojoListas.lock();
    try {
        listSalaDescansoV.add(id);
        gui.setSalaDescansoVendedores(listSalaDescansoV.toString());
    } finally {
        cerrojoListas.unlock();
    }
}

public void salirDescansoVendedor(String id) {
    cerrojoListas.lock();
    try {
        listSalaDescansoV.remove(id);
        gui.setSalaDescansoVendedores(listSalaDescansoV.toString());
    } finally {
        cerrojoListas.unlock();
    }
}

// Área de consumición, aforo máximo = 30
public void esperarMesa(String id){
    cerrojoListas.lock();
    try {
        listEsperandoConsumir.add(id);
        gui.setEsperandoConsumir(listEsperandoConsumir.toString());
    } finally {
        cerrojoListas.unlock();
    }
}

public void entrarConsumir(String id) throws InterruptedException {
    semAreaConsumicion.acquire(); // Espera hueco (30 max)
    cerrojoListas.lock();
    try {
        listEsperandoConsumir.remove(id);
        gui.setEsperandoConsumir(listEsperandoConsumir.toString());
        listConsumicion.add(id);
        gui.setAreaConsumicion(listConsumicion.toString());
    } finally {
        cerrojoListas.unlock();
    }
}

public void salirConsumir(String id) {
    cerrojoListas.lock();
    try {
        listConsumicion.remove(id);
        gui.setAreaConsumicion(listConsumicion.toString());
    } finally {
        cerrojoListas.unlock();
    }
    semAreaConsumicion.release();
}
}

```

```
// Métodos para RMI
public int getCantidadParque() {
    cerrojoListas.lock();
    try {
        return listParque.size();
    } finally {
        cerrojoListas.unlock();
    }
}

public int getCantidadCocina() {
    cerrojoListas.lock();
    try {
        return listCocina.size();
    } finally {
        cerrojoListas.unlock();
    }
}

public int getCantidadDescanso() { // Suma cocineros y vendedores
    cerrojoListas.lock();
    try {
        return listSalaDescansoC.size() + listSalaDescansoV.size();
    } finally {
        cerrojoListas.unlock();
    }
}

public int getCantidadAreaConsumicion() {
    cerrojoListas.lock();
    try {
        return listConsumicion.size();
    } finally {
        cerrojoListas.unlock();
    }
}
}
```

```
package pdp.practicacafeteria;

import java.util.ArrayList;
import java.util.concurrent.Semaphore;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

/**
 *
 * @author sergiopavon
 */
public class Despensa {
    private Log log;
    private InterfazCafeteria gui;

    // Stock
    private int cafes = 0;
    private int rosquillas = 0;
```

```

// Semáforos de aforo (en cocina: 50 cocineros y 50 vendedores)
private Semaphore aforoCocineros = new Semaphore(50);
private Semaphore aforoVendedores = new Semaphore(50);

private Lock cerrojo = new ReentrantLock();
private Condition hayComida = cerrojo.newCondition();

// Listas de cocineros y vendedores
private ArrayList<String> listCocineros = new ArrayList<>();
private ArrayList<String> listVendedores = new ArrayList<>();

public Despensa(Log log, InterfazCafeteria gui) {
    this.log = log;
    this.gui = gui;
}

// Operaciones de cocinero:
public void entrarCocinero(String id) throws InterruptedException {
    aforoCocineros.acquire();
    cerrojo.lock();
    try {
        listCocineros.add(id);
        gui.setDespensaCocineros(listCocineros.toString());
    } finally {
        cerrojo.unlock();
    }
}

public void dejarProductos(String id, int nCafes, int nRosquillas) {
    cerrojo.lock();
    try {
        this.cafes += nCafes;
        this.rosquillas += nRosquillas;

        log.escribirLog("Cocinero " + id + " deja " + nCafes + " cafés y " +
nRosquillas + " rosquillas en Despensa.");
        gui.setDespensaCafes(this.cafes);
        gui.setDespensaRosquillas(this.rosquillas);

        // Despertar a los vendedores que están esperando
        hayComida.signalAll();

    } finally {
        cerrojo.unlock();
    }
}

public void salirCocinero(String id) {
    cerrojo.lock();
    try {
        listCocineros.remove(id);
        gui.setDespensaCocineros(listCocineros.toString());
    } finally {
        cerrojo.unlock();
    }
    aforoCocineros.release();
}

// Operaciones de vendedor

```

```

    public void entrarVendedor(String id) throws InterruptedException {
        aforoVendedores.acquire();
        cerrojo.lock();
        try {
            listVendedores.add(id);
            gui.setDespensaVendedores(listVendedores.toString());
        } finally {
            cerrojo.unlock();
        }
    }

    // El vendedor intenta coger una determinada cantidad de productos
    public int[] cogerProductos(String id, int objCafe, int objRosquilla) throws
    InterruptedException {
        cerrojo.lock();
        try {
            // Si no hay nada, el vendedor espera
            while (this.cafes == 0 && this.rosquillas == 0) {
                log.escribirLog("Vendedor " + id + " esperando stock en
                Despensa...");
                hayComida.await();
            }

            int cCogidos = 0;
            int rCogidos = 0;

            // Coger cafés
            if (this.cafes >= objCafe) {
                cCogidos = objCafe;
                this.cafes -= objCafe; // Hay más cafes de lo que se piden
            } else {
                cCogidos = this.cafes; // Se cogen todos los cafés que quedan y
                Despensa vacía
                this.cafes = 0;
            }

            // Coger rosquillas
            if (this.rosquillas >= objRosquilla) {
                rCogidos = objRosquilla;
                this.rosquillas -= objRosquilla;
            } else {
                rCogidos = this.rosquillas;
                this.rosquillas = 0;
            }

            log.escribirLog("Vendedor " + id + " recoge " + cCogidos + " cafés y
            " + rCogidos + " rosquillas de Despensa.");
            gui.setDespensaCafes(this.cafes);
            gui.setDespensaRosquillas(this.rosquillas);

            // Array con la cantidad de cafés y rosquillas cogidos
            return new int[]{cCogidos, rCogidos};
        } finally {
            cerrojo.unlock();
        }
    }

    public void salirVendedor(String id) {
        cerrojo.lock();

```

```

        try {
            listVendedores.remove(id);
            gui.setDespensaVendedores(listVendedores.toString());
        } finally {
            cerrojo.unlock();
        }
        aforoVendedores.release();
    }

    // Métodos para RMI
    public int getCantidadCocineros() {
        cerrojo.lock();
        try {
            return listCocineros.size();
        } finally {
            cerrojo.unlock();
        }
    }

    public int getCantidadVendedores() {
        cerrojo.lock();
        try {
            return listVendedores.size();
        } finally {
            cerrojo.unlock();
        }
    }

    public int getStockCafes() {
        cerrojo.lock();
        try {
            return this.cafes;
        } finally {
            cerrojo.unlock();
        }
    }

    public int getStockRosquillas() {
        cerrojo.lock();
        try {
            return this.rosquillas;
        } finally {
            cerrojo.unlock();
        }
    }
}

```

```

package pdp.practicacafeteria;

import java.util.ArrayList;
import java.util.concurrent.Semaphore;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

/**
 *
 * @author sergiopavon
 */
public class Mostrador {

```

```

private Log log;
private InterfazCafeteria gui;

// Stock
private int cafes = 0;
private int rosquillas = 0;

// Aforos y colas
private Semaphore aforoVendedores = new Semaphore(20);

// ColaEsperaMostrador (máximo 20 esperando dentro de la cafetería).
// Los que no quepan aquí, se quedarán en Entrada Cafeteria
private Semaphore colaEsperaMostrador = new Semaphore(20);

// Max 5 clientes siendo atendidos
private Semaphore aforoAtencionClientes = new Semaphore(5);

private Lock cerrojo = new ReentrantLock();
private Condition hayStock = cerrojo.newCondition();

// Listas
private ArrayList<String> listVendedores = new ArrayList<>();
private ArrayList<String> listEntrada = new ArrayList<>(); // Nuevos:
Gente esperando entrar al local
private ArrayList<String> listClientesCola = new ArrayList<>(); // Gente
dentro (máximo 20)
private ArrayList<String> listClientesAtencion = new ArrayList<>(); // En
mostrador (máximo 5)

public Mostrador(Log log, InterfazCafeteria gui) {
    this.log = log;
    this.gui = gui;
}

// VENDEDOR
public void entrarVendedor(String id) throws InterruptedException {
    aforoVendedores.acquire();
    cerrojo.lock();
    try {
        listVendedores.add(id);
        gui.setMostradorVendedores(listVendedores.toString());
    } finally {
        cerrojo.unlock();
    }
}

public void reponer(String id, int nCafes, int nRosquillas) {
    cerrojo.lock();
    try {
        this.cafes += nCafes;
        this.rosquillas += nRosquillas;

        log.escribirLog("Vendedor " + id + " repone " + nCafes + " cafés y "
+ nRosquillas + " rosquillas en Mostrador.");
        gui.setMostradorCafes(this.cafes);
        gui.setMostradorRosquillas(this.rosquillas);

        // Despertar a clientes esperando stock
        hayStock.signalAll();
    }
}

```



```

    } finally {
        cerrojo.unlock();
    }
}

public void salirVendedor(String id) {
    cerrojo.lock();
    try {
        listVendedores.remove(id);
        gui.setMostradorVendedores(listVendedores.toString());
    } finally {
        cerrojo.unlock();
    }
    aforoVendedores.release();
}

// CLIENTE

// 1. Cliente nuevo, llega y entra en Entrada Cafeteria
public void entrarColaEspera(String id) throws InterruptedException {
    cerrojo.lock();
    try {
        listEntrada.add(id);
        gui.setEntradaCafeteria(listEntrada.toString());
    } finally {
        cerrojo.unlock();
    }

    // Intenta entrar a la colaEsperaMostrador (Semáforo de 20 huecos)
    // Si no hay hueco, se queda bloqueado aquí
    colaEsperaMostrador.acquire();

    // Consigue entrar: Pasa de la entrada a Esperando Mostrador
    cerrojo.lock();
    try {
        listEntrada.remove(id); // Sale de la puerta
        gui.setEntradaCafeteria(listEntrada.toString());

        listClientesCola.add(id); // Entra a la cola
        gui.setEsperandoMostrador(listClientesCola.toString());
    } finally {
        cerrojo.unlock();
    }
}

// 2. Pasar de la cola al mostrador (máximo 5)
public void accederMostrador(String id) throws InterruptedException {
    aforoAtencionClientes.acquire(); // Espera ser uno de los 5 atendidos

    cerrojo.lock();
    try {
        // Sale de la cola
        listClientesCola.remove(id);
        gui.setEsperandoMostrador(listClientesCola.toString());
        colaEsperaMostrador.release(); // Libera un hueco en la cola de 20
para el siguiente de la entrada

        // Entra al mostrador (lista ClientesAtencion)
        listClientesAtencion.add(id);

```

```

        gui.setEnMostradorClientes(listClientesAtencion.toString());
    } finally {
        cerrojo.unlock();
    }
}

// 3. Pedir y esperar stock
public void pedirYRecoger(String id, int demandaCafe, int demandaRosquilla)
throws InterruptedException {
    cerrojo.lock();
    try {
        while (this.cafes < demandaCafe || this.rosquillas <
demandaRosquilla) {
            hayStock.await(); // El cliente espera a que haya stock
disponible
        }

        // Preparar el pedido
        this.cafes -= demandaCafe;
        this.rosquillas -= demandaRosquilla;

        log.escribirLog("Cliente " + id + " toma su pedido. Quedan C:" +
this.cafes + " R:" + this.rosquillas);
        gui.setMostradorCafes(this.cafes);
        gui.setMostradorRosquillas(this.rosquillas);

    } finally {
        cerrojo.unlock();
    }
}

public void salirMostrador(String id) {
    cerrojo.lock();
    try {
        listClientesAtencion.remove(id);
        gui.setEnMostradorClientes(listClientesAtencion.toString());
    } finally {
        cerrojo.unlock();
    }
    // El cliente sale del mostrador y libera un hueco para que entre el
siguiente
    aforoAtencionClientes.release();
}

// Métodos para RMI
public int getCantidadVendedores() {
    cerrojo.lock();
    try {
        return listVendedores.size();
    } finally {
        cerrojo.unlock();
    }
}

public int getCantidadClientes() { // Suma de clientes en: cola + atendiendo
    cerrojo.lock();
    try {
        return listClientesCola.size() + listClientesAtencion.size();
    } finally {
        cerrojo.unlock();
    }
}

```

```

    }
}
public int getStockCafes() {
    cerrojo.lock();
    try {
        return this.cafes;
    } finally {
        cerrojo.unlock();
    }
}
public int getStockRosquillas() {
    cerrojo.lock();
    try {
        return this.rosquillas;
    } finally {
        cerrojo.unlock();
    }
}
}
}

```

```

package pdp.practicacafeteria;

import java.util.ArrayList;
import java.util.concurrent.Semaphore;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

/**
 *
 * @author sergiopavon
 */
public class Caja {
    private Log log;
    private InterfazCafeteria gui;

    private double recaudacionTotal = 0.0;

    // Aforo caja: 10 Clientes
    private Semaphore aforoCaja = new Semaphore(10);

    // Cerrojo para proteger variable compartida y listas
    private Lock cerrojo = new ReentrantLock();

    private ArrayList<String> listEsperandoCaja = new ArrayList<>(); // cola
    antes de coger máquina autopago
    private ArrayList<String> listEnCaja = new ArrayList<>(); // usando la
    máquina autopago

    public Caja(Log log, InterfazCafeteria gui) {
        this.log = log;
        this.gui = gui;
    }

    public void llegarColaCaja(String id) {
        cerrojo.lock();
        try {

```

```

        listEsperandoCaja.add(id);
        gui.setEsperandoCaja(listEsperandoCaja.toString());
    } finally {
        cerrojo.unlock();
    }
}

public void usarMaquinaPago(String id) throws InterruptedException {
    aforoCaja.acquire(); // Espera libre una de las 10 máquinas

    cerrojo.lock();
    try {
        listEsperandoCaja.remove(id);
        gui.setEsperandoCaja(listEsperandoCaja.toString());

        listEnCaja.add(id);
        gui.setEnCaja(listEnCaja.toString());
    } finally {
        cerrojo.unlock();
    }
}

public void confirmarPago(String id, double importe) {
    cerrojo.lock();
    try {
        recaudacionTotal += importe;
        gui.setRecaudacion(String.format("%.2f €", recaudacionTotal));
        log.escribirLog("Cliente " + id + " paga " + importe + "€. Total: " +
recaudacionTotal);

        // Sale de la caja
        listEnCaja.remove(id);
        gui.setEnCaja(listEnCaja.toString());
    } finally {
        cerrojo.unlock();
    }

    // Deja la máquina libre
    aforoCaja.release();
}

// --- MÉTODOS PARA RMI ---
public int getCantidadClientes() { // Cola + Pagando
    cerrojo.lock();
    try {
        return listEsperandoCaja.size() + listEnCaja.size();
    } finally {
        cerrojo.unlock();
    }
}

public double getRecaudacion() {
    cerrojo.lock();
    try {
        return this.recaudacionTotal;
    } finally {
        cerrojo.unlock();
    }
}
}

```

```
package pdp.practicacafeteria;

import java.util.Random;

/**
 *
 * @author sergiopavon
 */
public class Cocinero extends Thread {
    private String id;
    private ControlAforos aforos;
    private Despensa despensa;
    private InterfazCafeteria gui;
    private Random rand = new Random();

    public Cocinero(String id, ControlAforos aforos, Despensa despensa,
InterfazCafeteria gui) {
        this.id = id;
        this.aforos = aforos;
        this.despensa = despensa;
        this.gui = gui;
    }

    private void esperar(int min, int max) {
        try {
            gui.comprobarPausa();
            Thread.sleep(rand.nextInt((max - min) * 1000 + 1) + min * 1000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    public void run() {
        while (true) {
            try {
                gui.comprobarPausa();

                // 1. Sala descanso
                aforos.entrarDescansoCocinero(id);
                esperar(5, 10);
                aforos.salirDescansoCocinero(id);

                // 2. Trayecto cocina
                esperar(1, 3);

                // 3. Cocina
                aforos.entrarCocina(id); // Aforo máximo = 100

                // Preparar productos
                esperar(5, 10);

                boolean esCafe = rand.nextBoolean();
                int producidosC = 0;
                int producidosR = 0;
            }
        }
    }
}
```

```

        if (esCafe) {
            producidosC = rand.nextInt(4) + 2; // entre 2 y 5 cafés
        } else {
            producidosR = rand.nextInt(5) + 4; // entre 4 y 8 rosquillas
        }

        // Salir de la cocina y dirigirse hacia la despensa
        aforos.salirCocina(id);
        esperar(2, 5);

        // 4. Despensa
        despensa.entrarCocinero(id); // Aforo máximo = 50 cocineros

        despensa.dejarProductos(id, producidosC, producidosR);

        despensa.salirCocinero(id);

        // 5. Trayecto a sala de descanso
        esperar(2, 5);

    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
}
}
}

```

```

package pdp.practicacafeteria;

import java.util.Random;

/**
 *
 * @author sergiopavon
 */
public class Vendedor extends Thread {
    private String id;
    private ControlAforos aforos; // Solo para usar sala descanso visualmente si
    queremos
    private Despensa despensa;
    private Mostrador mostrador;
    private InterfazCafeteria gui;
    private Random rand = new Random();

    public Vendedor(String id, ControlAforos aforos, Despensa despensa, Mostrador
    mostrador, InterfazCafeteria gui) {
        this.id = id;
        this.aforos = aforos;
        this.despensa = despensa;
        this.mostrador = mostrador;
        this.gui = gui;
    }

    private void esperar(int min, int max) {
        try {
            gui.comprobarPausa();

```

```

        Thread.sleep(rand.nextInt((max - min) * 1000 + 1) + min * 1000);
    } catch (InterruptedException e) { e.printStackTrace(); }
}

@Override
public void run() {
    while (true) {
        try {
            gui.comprobarPausa();

            // 1. Sala Descanso
            aforos.entrarDescansoVendedor(id);
            esperar(5, 10);
            aforos.salirDescansoVendedor(id);

            // 2. Trayecto Despensa
            esperar(1, 3);

            // 3. Despensa
            despensa.entrarVendedor(id); // Max 50 vendedores

            int objCafe = rand.nextInt(4) + 3; // 3-6
            int objRosq = rand.nextInt(6) + 5; // 5-10

            // Coger productos (parcial o total)
            int[] cogidos = despensa.cogerProductos(id, objCafe, objRosq);

            // Preparar (1-3s) dentro o fuera? El PDF dice "Una vez
recopilados... se prepara en 1-3s"
            // Asumimos que esto ocurre antes de ir al mostrador, quizás aún
en despensa o saliendo.
            esperar(1, 3);

            despensa.salirVendedor(id);

            // 4. Trayecto Mostrador
            esperar(2, 5);

            // 5. Mostrador
            mostrador.entrarVendedor(id); // Max 20

            // Colocar productos (1-3s)
            esperar(1, 3);
            mostrador.reponer(id, cogidos[0], cogidos[1]);

            mostrador.salirVendedor(id);

            // 6. Trayecto Sala Descanso
            esperar(2, 5);

        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
}

```

```
package pdp.practicacafeteria;

import java.util.Random;

/**
 *
 * @author sergiopavon
 */
public class Cliente extends Thread {
    private String id;
    private ControlAforos aforos;
    private Mostrador mostrador;
    private Caja caja;
    private InterfazCafeteria gui;
    private Random rand = new Random();

    public Cliente(String id, ControlAforos aforos, Mostrador mostrador, Caja
caja, InterfazCafeteria gui) {
        this.id = id;
        this.aforos = aforos;
        this.mostrador = mostrador;
        this.caja = caja;
        this.gui = gui;
    }

    private void esperar(int min, int max) {
        try {
            gui.comprobarPausa(); // Verificar pausa antes de esperar
            Thread.sleep(rand.nextInt((max - min) * 1000 + 1) + min * 1000);
        } catch (InterruptedException e) { e.printStackTrace(); }
    }

    public void run() {
        try {
            gui.comprobarPausa();

            // 1. Entra al parque y lo contempla
            aforos.entrarParque(id);
            esperar(5, 10);

            // 2. Trayecto a cafetería
            aforos.salirParque(id);
            esperar(3, 9);

            // 3. Cola de espera mostrador
            mostrador.entrarColaEspera(id); // Se bloquea si hay 20 esperando

            // 4. Acceder al mostrador
            mostrador.accederMostrador(id); // Se bloquea si hay 5 en mostrador

            // Seleccionar pedido
            int nCafes = rand.nextInt(3) + 1;
            int nRosquillas = rand.nextInt(5);

            // Pedir y esperar stock
            mostrador.pedirYRecoger(id, nCafes, nRosquillas);

            // Salir del mostrador
            mostrador.salirMostrador(id);
        }
    }
}
```



```

        // 5. Caja
        caja.llegarColaCaja(id);

        caja.usarMaquinaPago(id); // Entra a la máquina
        esperar(2, 5); // Tiempo de pago

        double total = (nCafes * 1.50) + (nRosquillas * 2.50);
        caja.confirmarPago(id, total);

        // 6. Área de consumición
        aforos.esperarMesa(id);
        aforos.entrarConsumir(id); // Espera aforo 30

        esperar(10, 15); // Consumir

        aforos.salirConsumir(id);

    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
}

```

```

public class InterfazCafeteria extends javax.swing.JFrame {

    private boolean pausado = false;
    private final Lock cerrojoPausa = new ReentrantLock();
    private final Condition condPausa = cerrojoPausa.newCondition();

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    public InterfazCafeteria() {
        // Inicializa los componentes visuales (Código generado por el IDE)
        initComponents();

        // Configuraciones extra
        setTitle("Paradigmas de Programación - Simulación Cafetería");
        setSize(1250, 850);
        setLocationRelativeTo(null); // Centrar en pantalla
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    // Código de initComponents...
    // Control de Pausa
    public void cambiarEstadoPausa() {
        cerrojoPausa.lock();
        try {
            pausado = !pausado;
            if (pausado) {
                btnPausar.setText("REANUDAR");
            } else {
                btnPausar.setText("Pausar / Reanudar");
            }
        }
    }
}

```

```

        condPausa.signalAll();
    }
} finally {
    cerrojoPausa.unlock();
}
}

public void comprobarPausa() {
    cerrojoPausa.lock();
    try {
        while (pausado) {
            try {
                condPausa.await();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    } finally {
        cerrojoPausa.unlock();
    }
}

// Setters
public void setParque(String s) { txtParque.setText(s); }
public void setEntradaCafeteria(String s) { txtEntradaCafeteria.setText(s); }
public void setEsperandoMostrador(String s) {
txtEsperandoMostrador.setText(s); }
public void setEnMostradorClientes(String s) {
txtMostradorClientes.setText(s); }
public void setEsperandoCaja(String s) { txtEsperandoCaja.setText(s); }
public void setEnCaja(String s) { txtEnCaja.setText(s); }
public void setRecaudacion(String s) { txtRecaudacion.setText(s); }
public void setEsperandoConsumir(String s) { txtEsperandoConsumir.setText(s);
}

public void setAreaConsumicion(String s) { txtAreaConsumicion.setText(s); }

public void setSalaDescansoCocineros(String s) {
txtSalaDescansoCocineros.setText(s); }
public void setSalaDescansoVendedores(String s) {
txtSalaDescansoVendedores.setText(s); }
public void setCocina(String s) { txtCocina.setText(s); }
public void setDespensaCocineros(String s) { txtDespensaCocineros.setText(s);
}

public void setDespensaVendedores(String s) {
txtDespensaVendedores.setText(s); }
public void setDespensaCafes(int n) {
txtDespensaCafes.setText(String.valueOf(n)); }
public void setDespensaRosquillas(int n) {
txtDespensaRosquillas.setText(String.valueOf(n)); }
public void setMostradorVendedores(String s) {
txtMostradorVendedores.setText(s); }
public void setMostradorCafes(int n) {
txtMostradorCafes.setText(String.valueOf(n)); }
public void setMostradorRosquillas(int n) {
txtMostradorRosquillas.setText(String.valueOf(n)); }

// Variables declaration...

```

```

package pdp.practicacafeteria;

import java.io.FileWriter;
import java.io.PrintWriter;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

/**
 *
 * @author sergiopavon
 */
public class Log {
    private PrintWriter writer;
    private DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss");

    public Log() {
        try {
            writer = new PrintWriter(new FileWriter("evolucion_cafeteria.txt"));
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    // Synchronized para que solo un hilo pueda escribir a la vez
    public synchronized void escribirLog(String mensaje) {
        String tiempo = LocalDateTime.now().format(formatter);
        String linea = "[" + tiempo + "]" + mensaje;
        writer.println(linea);
        writer.flush();
    }
}

```

```

package pdp.practicacafeteria;

import java.rmi.Remote;
import java.rmi.RemoteException;

/**
 *
 * @author sergiopavon
 */
public interface ICafeteriaRemota extends Remote {

    public int obtenerClientesParque() throws RemoteException;
    public int obtenerClientesMostrador() throws RemoteException;
    public int obtenerClientesCaja() throws RemoteException;
    public int obtenerClientesConsumicion() throws RemoteException;

    public int obtenerCocinerosCocina() throws RemoteException;
    public int obtenerCocinerosDespensa() throws RemoteException;

    public int obtenerVendedoresDespensa() throws RemoteException;
    public int obtenerVendedoresMostrador() throws RemoteException;
}

```

```

    public int obtenerPersonalDescanso() throws RemoteException; // Suma de ambos

    public int obtenerStockDespensaCafe() throws RemoteException;
    public int obtenerStockDespensaRosquillas() throws RemoteException;
    public int obtenerStockMostradorCafe() throws RemoteException;
    public int obtenerStockMostradorRosquillas() throws RemoteException;

    public double obtenerRecaudacion() throws RemoteException;

    // Operación de control para pausar o reanudar el servidor
    public void pausarOReanudarServidor() throws RemoteException;
}

```

```

package pdp.practicacafeteria;

import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

/**
 *
 * @author sergiopavon
 */
public class CafeteriaRemotaImpl extends UnicastRemoteObject implements
ICafeteriaRemota {

    private ControlAforos aforos;
    private Despensa despensa;
    private Mostrador mostrador;
    private Caja caja;
    private InterfazCafeteria guiServer;

    public CafeteriaRemotaImpl(ControlAforos aforos, Despensa despensa, Mostrador
mostrador, Caja caja, InterfazCafeteria guiServer) throws RemoteException {
        super();
        this.aforos = aforos;
        this.despensa = despensa;
        this.mostrador = mostrador;
        this.caja = caja;
        this.guiServer = guiServer;
    }

    public int obtenerClientesParque() throws RemoteException {
        return aforos.getCantidadParque();
    }

    public int obtenerClientesMostrador() throws RemoteException {
        return mostrador.getCantidadClientes();
    }

    public int obtenerClientesCaja() throws RemoteException {
        return caja.getCantidadClientes();
    }

    public int obtenerClientesConsumicion() throws RemoteException {
        return aforos.getCantidadAreaConsumicion();
    }
}

```

```

    public int obtenerCocinerosCocina() throws RemoteException {
        return aforos.getCantidadCocina();
    }

    public int obtenerCocinerosDespensa() throws RemoteException {
        return despensa.getCantidadCocineros();
    }

    public int obtenerVendedoresDespensa() throws RemoteException {
        return despensa.getCantidadVendedores();
    }

    public int obtenerVendedoresMostrador() throws RemoteException {
        return mostrador.getCantidadVendedores();
    }

    public int obtenerPersonalDescanso() throws RemoteException {
        return aforos.getCantidadDescanso();
    }

    public int obtenerStockDespensaCafe() throws RemoteException {
        return despensa.getStockCafes();
    }

    public int obtenerStockDespensaRosquillas() throws RemoteException {
        return despensa.getStockRosquillas();
    }

    public int obtenerStockMostradorCafe() throws RemoteException {
        return mostrador.getStockCafes();
    }

    public int obtenerStockMostradorRosquillas() throws RemoteException {
        return mostrador.getStockRosquillas();
    }

    public double obtenerRecaudacion() throws RemoteException {
        return caja.getRecaudacion();
    }

    public void pausarOReanudarServidor() throws RemoteException {
        // Llamamos al método de la GUI del servidor que ya gestiona la pausa y
        // el botón
        guiServer.cambiarEstadoPausa();
    }
}

```